Edge online learning for river water level predictions

Omar Abozaid*, Félix-An Pham Ponton†, Meet Boghani† and Keyoumu Aisikeer†
*Department of Systems Engineering, École de technologie supérieure, Montreal, Canada
†Gina Cody School of Engineering and Computer Science, Concordia University, Montreal, Canada
Emails: omar.abozaid.1@ens.etsmtl.ca, f_phampo@live.concordia.ca, m_boghan@live.concordia.ca, ubcaskar@gmail.com

***Abstract*—Real-time river water level prediction is critical for disaster mitigation, yet traditional centralized hydrological models often struggle with high data transmission costs and the inability to adapt to sudden environmental changes. In this paper, we propose a lightweight *Edge Online Learning* framework designed to execute directly on river gauge stations. Unlike conventional offline models that prioritize generalization over historical data, our approach utilizes a rolling window normalization technique and a conditional training mechanism to continuously update the model parameters from streaming data. This allows the system to adapt rapidly to concept drift and sudden flood peaks without the need for massive historical storage or continuous cloud connectivity. We evaluate our framework using the Topľa river dataset, comparing it against standard offline deep learning architectures (LSTM, GRU, and CNN) and analyzing the impact of different adaptive optimizers (Adam, RMSProp, and a custom "Amnesia" strategy). Our results demonstrate that while offline models achieve lower global error through smoothing, the proposed online model exhibits superior sensitivity to extreme events, successfully capturing flood peaks that static models under-predict. This study highlights the potential of edge-based online learning as a responsive, resource-efficient solution for early warning systems.**

***Index Terms*—water level prediction, flood prediction, edge computing, online learning**

I. Introduction

Floods represent one of the most severe and recurrent natural hazards worldwide, causing extensive damage to critical infrastructure, disrupting socioeconomic activities, and undermining long-term environmental and societal stability [1], [2]. In many regions, repeated flooding events pose additional threats to agricultural sustainability and food security, as farming systems are highly sensitive to fluctuations in water availability [3], [4]. Sudden and extreme changes in river water levels can destroy cropland, damage irrigation networks, delay planting and harvesting schedules, and ultimately reduce crop productivity. Consequently, the ability to detect and forecast flood risks in a timely and reliable manner is essential for disaster mitigation, public safety, and sustainable regional development.

Traditional hydrological models—including rainfall-runoff simulations, conceptual and physically based watershed models, and statistical regression techniques—estimate flood risk by integrating meteorological inputs with hydrodynamic principles [5], [6]. Although such models provide interpretability, they often struggle to capture nonlinear dependencies and rapidly changing hydrological behaviors [7]. These limitations have become more pronounced as climate variability increases, leading to more frequent extreme rainfall events [8].

Driven by large-scale hydrometeorological datasets, machine-learning (ML) and time-series models have gained increasing attention [9], [10]. By leveraging precipitation, temperature, soil moisture, upstream flow data, and historical water-level observations, ML models can learn complex temporal dependencies and often outperform classical methods [11]. Deep-learning architectures such as recurrent neural networks (RNNs), Long Short-Term Memory networks (LSTMs), and Transformer-based models further enhance the modeling of long-range hydrological patterns [12]–[14].

Despite the success of these deep learning architectures, their deployment in real-world hydrological monitoring systems faces significant hardware constraints. State-of-the-art models often require substantial computational power and memory that exceed the capacities of standard edge computing devices deployed at remote river gauge stations. This has led to the growing interest in efficient Edge AI solutions [15], which aim to execute machine learning tasks locally on the edge. However, standard edge approaches primarily focus on static model compression for inference and do not address the need for continuous on-device training to handle evolving hydrological patterns.

Most existing hydrological forecasting systems rely heavily on centralized designs and offline learning [16]. In these systems, large amounts of data are collected from many different locations and processed in a central server to build huge training datasets. This approach requires expensive infrastructure for data transmission and storage, which is often difficult for developing countries to maintain. Furthermore, offline models cannot easily adapt to new conditions or rapid changes in river levels [17].

To solve this, we propose an edge computing solution where learning happens directly at the gauge station. By using online learning, the model updates itself as new data arrives, allowing it to adapt continuously to changing river conditions [18], [19]. While this method is common in finance and robotics [20], it is not widely used for water-level prediction. This study investigates how effective online learning is for this purpose.

all the implementation details and code are available through github link: https://github.com/omardwahba/water_lvl_pred

The main contributions of this paper are:

- We propose a lightweight online learning framework for edge devices that uses rolling window normalization to handle changing data patterns without needing to store large amounts of historical data.
- We compare different adaptive learning rate optimizers and introduce a dynamic mechanism to help the model react faster to sudden flood peaks compared to standard offline methods.

The rest of this paper is organized as follows: Section II reviews related work. Section III explains our methodology. Section IV describes the experiments. Section V presents the results, and Section VI concludes the paper.

II. Related work (literature review)

This section provides an overview of the machine learning approaches that have been applied to flood prediction. We examine a range of models, highlighting each study's methodological contribution, datasets and performance. By comparing these works, we aim to identify key trends, strength and limitations in current ML-based flood prediction models. We survey recent works on fully connected networks (MLP) and recurrent models like LSTM.

A. Multilayer Perceptron (MLP)

Thank to its simplicity, the multilayer perceptron has been widely used as an early machine learning approach to flood prediction, particularly for rainfall-runoff and daily water level prediction tasks [21] [22]. Their feed-forward structure makes them suitable for modeling general nonlinear relationships when temporal dependencies are not explicitly considered.

Riad et al (2004) [21] modeled the rainfall-runoff relationship for the Ourika basin in Morocco using an MLP trained via backpropagation. Their network made use of rainfall and runoff values from the previous seven (7) days to predict the next day's discharge. Discharge is an early and reliable indicator of upcoming flood condition because it indicate how much water is moving through the river and flood occurs when the river's flow exceed the channel capacity. The database compiled represents seven years (1990 to 1996) of daily sets of rainfall-runoff values for the basin. They used the data from the six first year (1990-1995) for model training and the last year (1996) for testing. They demonstrated that the MLP can accurately captured nonlinear behavior, achieving strong agreement between predicted and observed flows value with a $R^2 = 0.917$ in the testing phase.

A more extensive study into MLP design choices was conducted by Rezaeian Zadeh et al (2010) [22], who focused on daily outflow prediction in the Khosrow Shirin watershed in Iran. The dataset used was comprised of a daily set of precipitation and river flow data taken from multiple station on the watershed. Their study systematically compared five different MLP input configurations and evaluated two activation functions, logistic and tangent sigmoid, using the Levenberg-Marquardt algorithm. They concluded that the choice of input vector has a significant impact on the prediction accuracy. Also, they showed that models involving antecedent precipitation and discharge better predict daily flow. Interestingly, they showed that the best input model uses precipitation at time t and t-1 together with the discharge at t-1 consistently produced the best results and that adding more input variables (e.g., adding precipitation at time t-2) did not necessarily improve performance.

These studies illustrate the strengths of MLP for the flood-related prediction task. Strong performance with relatively simple architecture.

B. Long Short-Term Memory (LSTM)

Recurrent neural network architecture, particularly those with gating mechanisms such as the Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU), has emerged as leading approaches for flood and water-level forecasting. This is due to their capacity to model temporal dependencies and sequential dynamics that traditional neural networks struggle with, enabled by their gating mechanism that allows the model to retain information over extended sequences.

Le et al (2023) [23] uses an LSTM model to forecast multiple-day (1,2 and 3 day) flood discharges at the Hoa Binh Station on the Da River in Vietnam. They assessed the model's ability in flood forecasting and the effects of input data characteristics on model performance. Their results show that the LSTM model achieved very high predictive performance across all forecasting horizons. During validation, the model produced NSE values of 99%, 95%, 87 for one-day, two-day, three-day forecasts respectively, indicating strong ability to capture both peak flow and temporal discharge. They also show the model ability to capture extreme event, as the LSTM successfully reproduced the 1984 flood peak with relative errors of just 5% for the one-day forecast and 14% for the two-day forecast.

In their study, Zakaria et al. (2023) [24] compared MLP and LSTM models for daily water-level forecasting on the Muda River in Malaysia. They used a three-year dataset of water-level and meteorological observations. They tested both univariate and multivariate input configurations to assess how different input combinations influence predictive skill. The authors found that, for one-day-ahead prediction, MLP achieved the highest overall accuracy ($R^2 = 0.871$), outperforming the LSTM. However, for longer forecasting horizons, up to seven days ahead, the LSTM demonstrated superior performance.

C. Hybrid Models

Recent Studies have focused on hybrid deep learning architectures that combine multiple neural network components to better capture the complexity of flood-related processes. A particular hybrid model is the Hydro-Informer, it combine CNN, LSTM, GRU, and Multi-Head Attention mechanism to predict water level and extreme hydrological events with high reliability (peak prediction). The model was introduced by Almikaeel et al (2024) [25] to address limitation of traditional LSTM configuration regarding extreme event detection [26].

Ref.	Model	Input Features	Output Features	Pred. Type	Brief Methodology	Region	Design
[21]	MLP	Daily rainfall and discharge from a single station	Discharge	Daily	Compared MLP to traditional regression models.	Morocco	Offline / Centric
[22]	MLP	Daily average rainfall and average discharge from multiple stations	Discharge	Daily	Evaluated several input combinations and activation functions for MLP.	Iran	Offline / Centric
[23]	LSTM	Daily rainfall and discharge from multiple stations	Discharge	Daily	Forecasted 1–3 days ahead using LSTM for multistep discharge prediction.	Vietnam	Offline / Centric
[24]	MLP vs LSTM	Water level and meteorological data (temperature, pressure, humidity)	Water level	Daily	Compared MLP and LSTM using identical input scenarios.	Malaysia	Offline / Centric
[25]	CNN + LSTM/GRU + Multi-head Attention	Water level, rainfall, and discharge	Water level	Hourly	Trained a hybrid CNN–RNN–Attention model to produce 12-hour water-level forecasts, with emphasis on peak-level detection.	Slovakia	Offline / Centric

TABLE I: Comparison of machine learning models used for flood prediction in the reviewed literature.

The Hydro-Informer was trained and tested on a 12-year hourly dataset (2008-2012) including water level, discharge, and rainfall measurements as inputs features from the Topl'a River in Slovakia. They used 36 hours of historical inputs features to predict the next 12 hours of water levels. The hybrid architecture leverages CNN layers to extract short-term temporal patterns, LSTM/GRU layers to understand sequences of data over time, helping the model predict future conditions, and the multi-head attention to focus on critical features associated with rising water levels. All crucial components are for robust hydrological forecasting. A key component of the model is the custom loss function that assigns higher penalties to errors during extreme water-level conditions ensuring the model emphasizes correct peak prediction

The results show Hydro-Informer achieving strong performance ($R^2 = 0.88$ and RMSE of 4,25cm), outperforming simpler deep models in capturing extreme event. A notable characteristic demonstrated in testing by predicting major peaks 1-5 hours in advance with high accuracy in both timing and magnitude, an essential capability for operational flood warning systems.

Overall, the studies reviewed (Table 1) demonstrate that offline machine-learning models can achieve strong predictive performance for flood and water-level forecasting, particularly when provided with well-structured historical time-series inputs. Across the models reviewed, a key trend is that the temporal structure of the data, lagged water-level or discharge sequences, is the primary driver of forecasting accuracy, often outweighing the influence of additional meteorological variables. They also follow, for the most part, a centric, offline design, where models are trained once on a fixed historical dataset, then deployed without continuous updating. Limiting their adaptability in rapidly evolving hydrological contexts. Therefore, online machine learning presents a viable approach to mitigating these limitations. Online learning has been shown to accurately predict time sequences. [27] Also, online learning continuously updates model parameters as new observations arrive, allowing the model to adapt to evolving patterns, especially for extreme event (e.g. peak water-level) as they are often underrepresented in historical datasets. They are also well-suited for streaming sensor environments [28] permitting the use of edge devices for real-time flood monitoring.

III. Methodology

In this section, we explain the design and steps of our edge online learning framework for water level predictions. In this section, we discuss two main points: 1) What's the motivation behind using online learning and edge computing strategies? 2) Our framework description and different parameters.

A. Edge Online learning solution

We propose a new framework for water level prediction based on online machine learning for edge devices, which is inspired from TinyML [15]. Instead of depending on a centric design that collects the data from different gauge locations and processes it in one place. Therefore, the centric design increases the cost of data collection, transmission, and processing. In addition to that, more features mean a more complex pattern that the machine learning models should be able to detect and recognize. Therefore, the models that are produced by centric design are usually massive (huge trainable parameters- big size) and require significant resources [15]. To mitigate this, we propose an edge solution that moves data processing to the gauge station (see fig 1. By moving data processing closer to the source, real-time predictions are facilitated. Additionally, this significantly reduces the cost of data transmission, as only the processed data will be sent for further operations, such as long-term predictions. The edge framework focuses on short-term predictions (in hours) as it is more suitable for gauge stations hardware, which usually relies on small embedded system devices. This unlocks the

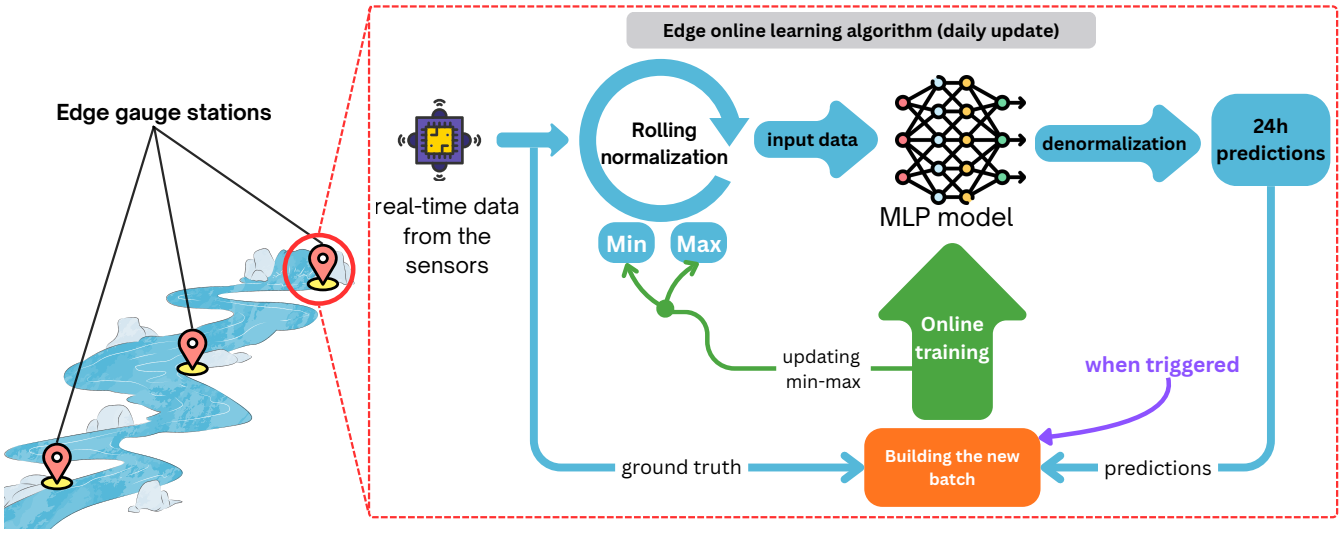


Fig. 1: Our proposed Edge online framework for 24h prediction

potential to add more gauge points as hardware resources and communication are feasible.

The second problem we are addressing is the nature of floods themselves. Floods are an unexpected increase in river water levels. In Fig 2 we can spot floods as the spikes in the data. Those spikes represent sudden events that appear in the data distribution. Machine learning models always try to generalize over the data distribution; therefore, they cannot predict well the sudden change, as it is not a normal (general) condition. The other problem that traditional offline-trained models suffer from is concept drift. In a dynamic environment like rivers, the data's pattern can deviate from what the model was originally trained on, causing accuracy decay as the model needs to be fine-tuned to cope with the new data pattern. To address this issue, we propose the use of online learning to tackle the following:

1) Cope with the dynamic environment: Offline training typically works in a two-way fashion. First, the model is trained on the training dataset and evaluated on the testing dataset. Once the desired performance is reached, the model is then deployed to be used for inference (predictions). To fine-tune the model, you need to do the whole cycle again, plus the data collection part. Online learning facilitates training the model on-the-fly from a data stream [29]. Online learning splits the dataset into patches and updates the model on each patch. This facilitates fine-tuning the model training on a data stream, as future collected data is a batch; once the batch is collected and the update is triggered, online learning trains the model. This design enables the model to adapt to the pattern shift and solve the concept drift problem.

2) Cope with limited resources: since we are adopting an edge computing approach, where hardware resources are limited. Online learning helps reduce the cost required for training, unlike traditional offline learning, which requires the presence of the whole dataset in memory; online learning only needs to preserve the batch size, reducing the memory requirements for training. Also, online learning facilitates speeding up the training process by reducing the `Batch_size` and limiting the number of epochs per batch. This technique trades the time required for learning (number of epochs per batch) with the number of batches, meaning that the model is getting better the more batches come, and for every batch we have a lower processing time, as we don't go back and forth with the same batch.

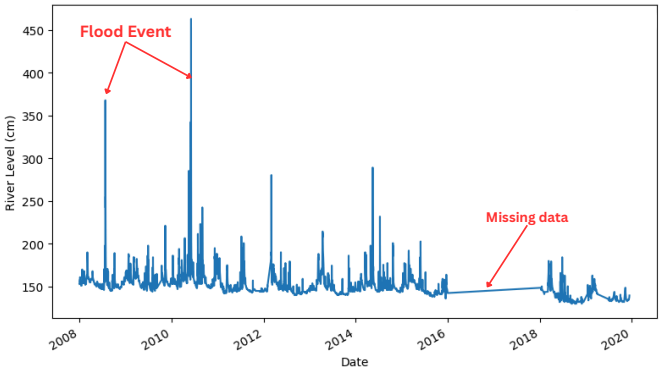


Fig. 2: Topľa River water level on the full dataset

B. Framework description

Figure 1 illustrates the complete workflow of our edge online learning system. The process operates in a cycle at the gauge station level, divided into two main phases:

1) Real-time Inference (Top Path): The system collects real-time data from the sensors. This data first passes through a **Rolling normalization** block, which dynamically scales the input using the minimum and maximum values of the current window. The normalized input data is then fed into the MLP

model, which generates a forecast. Finally, a **denormalization** step converts the output back to the original scale (e.g., cm) to produce the 24h predictions.

2) Conditional Online Training (Bottom Path): Unlike continuous learning systems that update with every new sample, our framework implements a controlled update strategy. As indicated by the "when triggered" signal in the figure, the training loop is activated selectively. When enabled, the system collects the observed water levels (**ground truth**) for the predicted period and combines them with the predictions in the **Building the new batch** block. The **Online training** module then uses this batch to execute a single optimization step, updating both the model's weights and the rolling normalization parameters (min-max). This controllable mechanism ensures the model can adapt to changing environments while allowing for resource conservation when updates are not required.

To support this workflow, we frame the problem as a multivariate time-series prediction task where we consider short-term predictions only. Short-term predictions allow precise monitoring of the river water level. We assume hourly predictions as we don't have the resources and capacity for long-term historical data processing. We use 48 hours of historical data from available features, plus historical river water level data, to predict the next 24 hours. In other words, our `lookback_value` $= 48$ and our `Horizon` $= 24$. We propose those values as a sweet spot to balance between the memory requirement to store historical data and the future horizon that gives a practical early warning for the authorities. The way we implemented online learning on timeseries forecasting using a sliding window approach, where the window is shifted with the `Horizon` value instead of going one point at a time. This modified sliding window technique reduces the time and resource consumption for training as each point is only fed to the model one time. So our framework will wait for 24 points, then calculate the losses and optimize the model. In addition, we choose to use an MLP as a simple neural network that can be optimized later for constrained resources using TinyML techniques [15].

Since features are measured in different ranges, we need to normalize the data so that the model is not biased to a certain feature. However, traditional normalization techniques, such as min-max normalization, assume global knowledge of the data means; they determine the global max and global min, or the global variance of all the data. However, this assumption can't hold in a data-stream case, as you don't know what the upcoming data will be. In our framework, we adopted a rolling window normalization technique where we keep track of min/max values per feature for each batch. The batch is normalized before it gets fed to the model, then the output of the model and the loss are calculated then the output is denormalized back to the original value using the min/max values recorded before. Therefore, Window normalization overcomes the global normalization assumption that the time-series data is stationary (has static constant statistical properties, i.e, variance, min, max, etc.) [30].

Another key in our framework is the **optimizer**. Choosing the right optimizer is crucial, especially when we are using online learning, as we assume a data stream of batches, and we only go for one optimizer step per batch; we need an optimizer that can converge the model as soon as possible. Another problem with the optimizers is the drift and changes in the data. Since floods are exceptional events in the water level, we needed an optimization that adapts and recognizes these special events so that the model can follow the environmental changes and the data drift. We conduct several studies on recent optimizers usually used to optimize deep learning models and report our findings in section V.

Algorithm.1 shows our online learning implementation, starting from building the dataset to mimic the data stream condition. We start by looping through the whole data to build the input and output for each batch. The input represents the `lookback_value` (48) of the historical features as input, then attaches it to the future target feature (water level) as the ground truth for this input. After building the batch x_batch and y_batch, they get normalized using the rolling normalization technique, where only the min/max is calculated from the input features (known values). Then, the normalized data is fed to the model for training. We also limit the optimizers' steps to only (1) to reduce training cost. Finally, the output of the model is denormalized for the metric evaluations.

IV. Experimental study

To test our approach, we carefully design our experiments with the following objectives. Firstly, we need our model to be able to recognize the peaks that represent an expected flood. That's why we focus on short-term predictions, as this is where our edge computing approach is needed. Moreover, we prefer model adaptability to special events over good predictions under normal conditions. We care about parameters like optimizers and learning rate that don't hold down the model when it tries to adapt to a strong peak (special event). This section is divided into two parts. Part I: the case study and dataset description. Part II: illustrates our experimental setup and testing strategy to ensure the objectives mentioned above. Part III: shows the models' architecture used in our experiments.

A. Dataset

We utilized the publicly available Topľa River dataset provided by [25]. Spanning from 2008 to 2019, the dataset includes hourly measurements from four locations; however, our study restricts data to the `Bardejov` station to simulate a single-edge node constraint. The data captures two major flood events (July 2008, June 2010) and contains a recording gap from 2016 to 2018 (see Fig. 2). The `Bardejov` station records four key features used for our multivariate analysis:

- **Precipitation (Q):** Measured in mm.
- **Temperature (T):** Measured in $^{\circ}C$.
- **Discharge rate (Q):** Measured in m^3/s.
- **Water level (H):** Measured in cm.

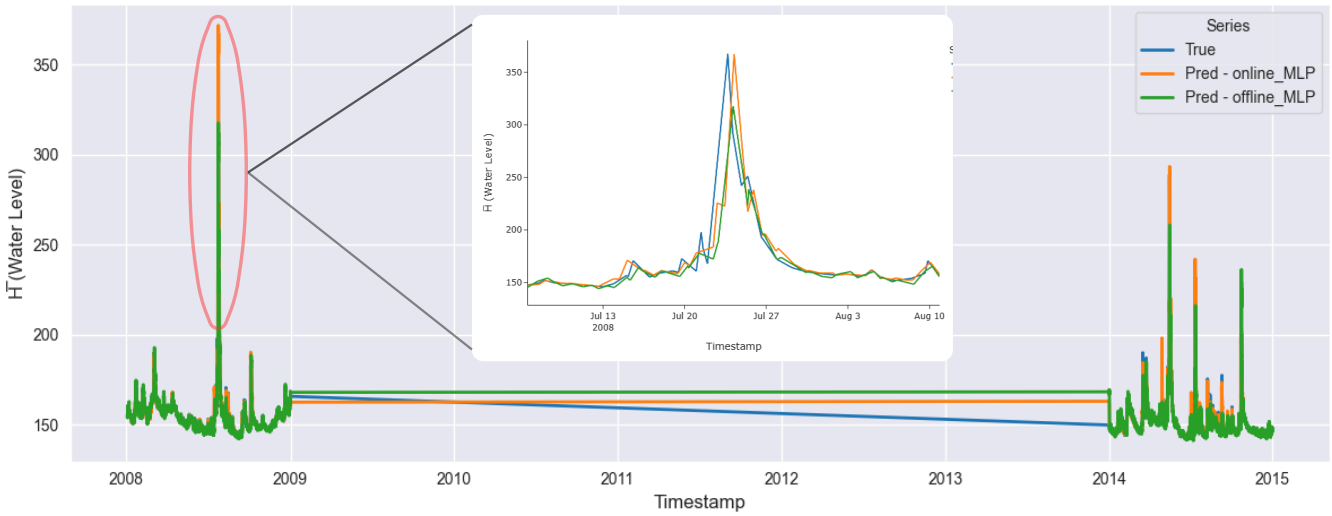


Fig. 3: Online vs offline learning using MLP model with zoom on 2008 flood event

Algorithm 1 Online Time-Series Forecasting with Rolling Normalization

Input:

Data stream $\mathcal{D} = \{d_1, d_2, \ldots, d_T\}$
Lookback L, Horizon H
Model f_θ, Optimizer Opt, Loss $\mathcal{L}$
Feature index idx, stability constant ϵ

Output: Optimized parameters θ, Predictions $\hat{Y}$

Initialize θ randomly
$t \leftarrow 0$
while $t + L + H \leq T$ **do**
  // 1. Sequence Extraction
  $\mathbf{X}_t \leftarrow [d_t, \ldots, d_{t+L-1}]$
  $\mathbf{y}_t \leftarrow [d_{t+L}[idx], \ldots, d_{t+L+H-1}[idx]]$
  // 2. Rolling Normalization
  $X_{min} \leftarrow \min(\mathbf{X}_t)$, $X_{max} \leftarrow \max(\mathbf{X}_t)$
  $R_X \leftarrow X_{max} - X_{min} + \epsilon$
  $\tilde{\mathbf{X}}_t \leftarrow (\mathbf{X}_t - X_{min})/R_X$ ▷ Normalize Input
  $y_{min} \leftarrow \min(\mathbf{y}_t)$, $y_{max} \leftarrow \max(\mathbf{y}_t)$
  $h_{min} \leftarrow \min(X_{min}[idx], y_{min})$
  $h_{max} \leftarrow \max(X_{max}[idx], y_{max})$
  $R_h \leftarrow h_{max} - h_{min} + \epsilon$
  $\tilde{\mathbf{y}}_t \leftarrow (\mathbf{y}_t - h_{min})/R_h$ ▷ Normalize Target
  // 3. Online Inference & Training
  $\hat{\mathbf{y}}_{scaled} \leftarrow f_\theta(\tilde{\mathbf{X}}_t)$ ▷ Forward Pass
  $Loss \leftarrow \mathcal{L}(\hat{\mathbf{y}}_{scaled}, \tilde{\mathbf{y}}_t)$
  $Opt.zero\_grad()$
  $\nabla_\theta \leftarrow \frac{\partial Loss}{\partial \theta}$ ▷ Compute Gradients
  $\theta \leftarrow Opt.step(\theta, \nabla_\theta)$ ▷ Update Weights
  // 4. Reconstruction (Optional)
  $\hat{\mathbf{y}}_{final} \leftarrow (\hat{\mathbf{y}}_{scaled} \times R_h) + h_{min}$
  $t \leftarrow t + 1$ ▷ Move to next time step
end while

B. Experimental setup

To test the performance of our online learning approach, we perform a comparative study between online learning and offline learning. In the first experiment, the MLP model is trained on the dataset in both online and offline fashion using the same parameters and optimizers, then both models are evaluated on the testing dataset. The second experiment includes a comparison between our online learning model and some other known traditional and deep learning models that are widely used in time-series forecasting, and compares the trade-offs between peak value analysis and overall metrics.

Optimizer Selection strategy: A critical challenge in online learning is the learning rate. A low rate causes slow adaptation to rapid flood peaks, while a high rate introduces noise and can cause the model to miss the special event. To address this, in our third experiment, we compare several adaptive learning rate optimizers: **Adam** [31], which combines momentum and RMSProp [32] scaling; **AdamW** [33], which decouples weight decay; **RMSProp**, known for efficacy in non-stationary settings; and **RAdam** [34], which stabilizes early variance.

Additionally, we introduce a custom approach called **"Amnesia"**. This technique combines a *resetting mechanism* [35] that clears the optimizer's internal buffers (momentum and variance) with a *warm restart* [36]. The hypothesis is that during high-loss events (floods), resetting the internal state allows the model to "forget" outdated stationary dynamics and adapt faster to the new regime. The Amnesia approach relies on three hyperparameters: 1) A **sensitivity threshold** which acts like a trigger; it determines how much larger the current error must be compared to the average to be considered a 'special event' rather than just normal noise. 2) A **warmup period** which serves as a safety delay to ensure the model has time to stabilize initially before we start monitoring for changes. 3) A **restart learning rate** which is the temporarily increased speed applied immediately after a reset to help the

model rapidly learn the new pattern before settling back to normal.

C. Models Descriptions

*1) **MLP (Multi-Layer Perceptron)**:* A feed-forward dense neural network consisting of layers **128** $\rightarrow$ **64** $\rightarrow$ **24**, each followed by ReLU activations. The model flattens the 48-hour lookback window so that each prediction is made from a single vector of past features. MLPs are simple, fast to train, and serve as a strong baseline when temporal patterns can be encoded through feature engineering instead of recurrence.

*2) **Logistic Regression**:* A linear model with a single dense layer mapping the flattened lookback vector to the 24-hour prediction horizon. It models the target as a direct linear combination of past features. This provides a useful comparison point for understanding how much nonlinear capacity is required for accurate water-level forecasting.

*3) **LSTM (Long Short-Term Memory)**:* A recurrent neural network composed of two stacked LSTM layers feeding into a final linear output layer. LSTMs are designed to capture long-range temporal dependencies through gated memory cells, making them effective when water-level dynamics depend on patterns spanning multiple hours or days.

*4) **GRU (Gated Recurrent Unit)**:* Similar in purpose to LSTMs but using fewer gates. GRUs maintain temporal dependencies through update and reset gates, reducing parameter count and computational cost. They frequently match or outperform LSTMs on sequence tasks while being more efficient.

*5) **Conv-1D (One dimension Convolution)**:* A temporal CNN applying 1D convolutions along the time axis, followed by average pooling and a final linear layer. CNNs extract local temporal features, such as short-term spikes or fluctuations, and are computationally efficient due to weight sharing across time steps.

V. Results

In this section, we illustrate our results and the conclusions drawn from the interpretations of the results. We start by reporting the first comparison between online and offline training using the same model. Then, we compare the performance of our model vs different offline model architectures that are used for time-series forecasting. Finally, we report the last experiment where we compare using different adaptive learning rate optimizers.

A. online vs offline using same model (MLP)

For the **offline training:** we trained the models on a globally normalized training set, without shuffle (as it's sequential data), and epochs that run until *early stopping* of 10 epochs. Each architecture sees all historic samples multiple times. For the **online training:** we update the model sequentially on rolling windows without revisiting past batches, using window normalization. After the training, both models' parameters are fixed and then evaluated on the test set. In addition we perform a peak analysis around the flood that happened in 2008 (included in test set). Both techniques use the proposed MLP model with the same parameters. Moreover, for both methods, we shift the prediction window by the `Horizon` value to avoid many repetitive points.

Table II indicates that the offline MLP model outperforms the online version in global metrics, achieving a lower RMSE (4.40 vs 5.11). This is expected, as the offline model minimizes error over the entire dataset history, favoring smooth, generalized predictions. However, peak analysis (Fig. 3) reveals a crucial trade-off. The offline model's smoothing effect causes it to under-predict the sharp 2008 flood peak. In contrast, the online model, which continuously updates parameters with incoming data, exhibits superior sensitivity to non-stationary behaviors. By prioritizing recent information, it successfully tracks the rapid rise in water levels, proving more effective for real-time flood warning despite the slightly higher global noise.

B. Comparison with Standard Offline Architectures

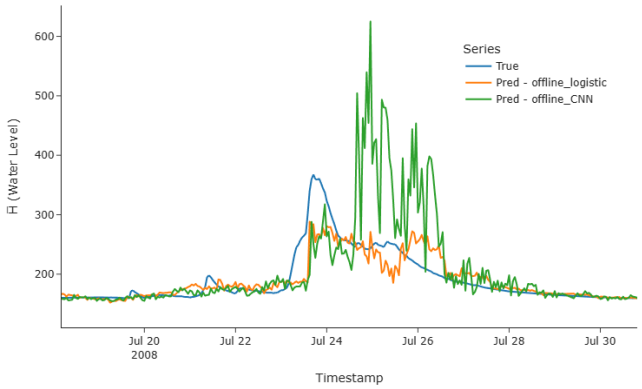
To further evaluate the effectiveness of our approach, we compared our Online MLP model against widely used time-series architectures trained in a traditional offline manner. This experiment aims to understand where our adaptive model stands compared to established deep learning models like CNNs, LSTMs, and GRUs.

Figure 4 illustrates the performance of these different architectures. We first examine the performance of Logistic Regression and ID-Convolutional Neural Networks (CNN) (see Fig. 4-a). As shown in the plot, the Logistic Regression model acts as a baseline but fails to capture complex nonlinear behaviors. On the other hand, the CNN model exhibited high instability. While CNNs are excellent at feature extraction, in this specific hydrological context, the model became excessively noisy, over-reacting to local fluctuations rather than capturing the true trend of the water level. Due to this poor performance and instability, we exclude CNN and Logistic Regression from further detailed peak analysis.

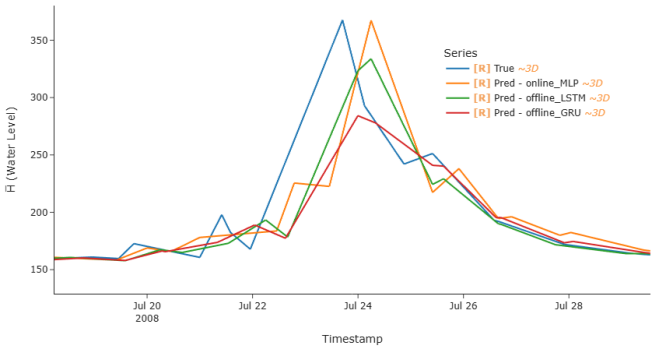
We narrow our focus to the Recurrent Neural Networks (LSTM and GRU), which are considered standard benchmarks for time-series forecasting (see Fig. 4-b). The results reinforce the conclusion drawn in our first experiment: offline learning prioritizes generalization. The offline LSTM and GRU models produce smooth curves that follow the general trend of the river very well. However, this smoothness comes at a cost. Because these models are trained offline on the entire history, they are biased toward the "average" river conditions. Furthermore, the recurrent mechanisms (gates) in LSTMs and GRUs are designed to retain long-term memory. In the context of

TABLE II: Offline and online model performance comparison

Mode	Model	Params	RMSE	MAE	R^2
Offline	MLP	34,520[1]	4.4034	1.8619	0.8533
Online	MLP (ADAM)	34,520[1]	5.1112	1.4376	0.8024
Offline	LSTM	52,760[2]	4.5624	1.6675	0.8425
Offline	GRU	39,960[3]	4.2986	1.4262	0.8602
Offline	CNN	39,960[3]	6.1587	2.8862	0.7130



(a) Performance of Offline CNN and Logistic Regression.



(b) Performance of Offline LSTM and GRU vs Online MLP.

Fig. 4: Comparative analysis of different model architectures. (a) shows the instability of CNN and the limitations of Logistic Regression. (b) shows that while offline LSTM and GRU offer smooth generalized predictions, the Online MLP adapts better to the peak flood intensity.

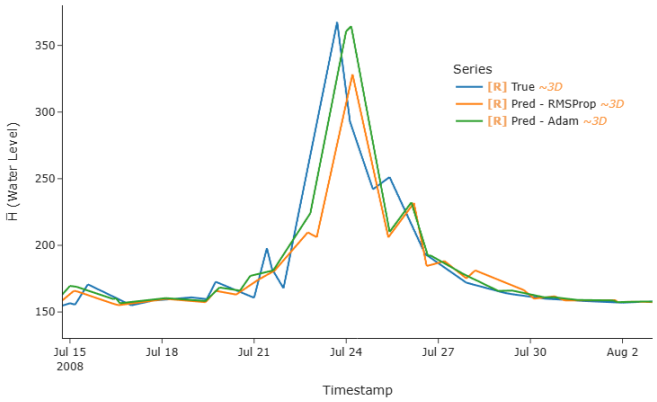
a sudden flash flood, this memory acts as a buffer, making the model "stiff" or slow to react to the rapid state change. In contrast, our Online MLP, despite being a simpler architecture, demonstrates superior adaptability. By updating its weights with every new batch, it is not held back by the memory of previous stationary states, allowing it to spike immediately alongside the flood event.

C. Comparison of Adaptive Optimizers

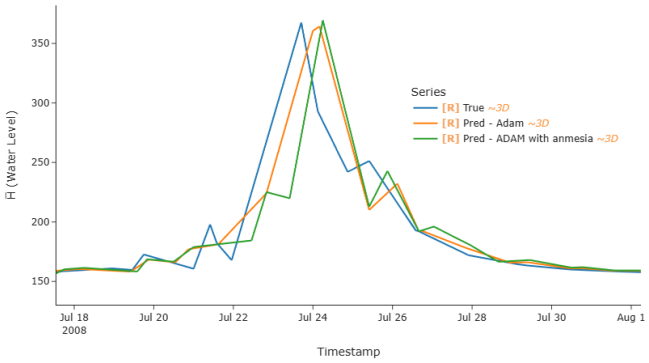
In our third experiment, we evaluated the stability versus adaptability trade-off across the selected optimizers. All optimizers were initialized with a fixed learning rate of $\eta = 0.001$, keeping the model architecture consistent. The quantitative results are summarized in Table III. Among the standard optimizers, **RMSProp** demonstrated superior stability, achieving the lowest RMSE (4.208) and highest R^2 (0.866). This suggests that RMSProp's moving average of squared gradients effectively minimizes global error in this hydrological time series. In contrast, Adam and its variants (AdamW, RAdam) yielded higher error rates (RMSE $\approx$ 5.0), indicating that the momentum component, while aiding convergence, may introduce additional noise during stable periods.

TABLE III: Performance comparison of different adaptive optimizers on the test dataset.

Optimizer	**RMSE**	**MAE**	R^2
Adam	4.993	1.404	0.811
AdamW	5.048	1.425	0.807
RMSProp	**4.208**	**1.243**	**0.866**
RAdam	5.267	1.495	0.790
Adam with Amnesia	5.830	1.629	0.743



(a) Comparison of RMSProp vs Adam.



(b) Comparison of Adam vs Adam with Amnesia.

Fig. 5: Visual comparison of online learning performance. (a) shows Adam's superior peak tracking versus RMSProp's stability. (b) compares the baseline Adam versus the experimental Amnesia strategy.

However, global metrics such as RMSE do not tell the full story regarding the model's ability to capture critical events. As illustrated in Figure 5-a, a visual inspection of the flood peak reveals a trade-off between stability and sensitivity. While RMSProp achieved better overall metrics, it struggled to capture the flood event compared to Adam. The Adam optimizer, despite its higher overall noise (reflected in the higher RMSE), exhibited higher sensitivity to gradients, allowing it to adapt more rapidly to the rising limb of the flood and predict a peak

value much closer to the ground truth.

Finally, we evaluated the proposed "Amnesia" strategy (Figure 5-b). While the quantitative results for *Adam with Amnesia* were lower than the baselines (RMSE 5.8300), the experiment highlights a potential avenue for improvement. The current implementation relies on a fixed threshold ($2.5\times$ running average loss) to trigger the reset. The results suggest that while the current sensitivity creates instability, the mechanism itself is optimizer-agnostic. It shows potential to be refined or combined with more stable optimizers like RMSProp to balance the trade-off between the high stability of RMSProp and the rapid peak adaptation required for flood forecasting.

In conclusion, while RMSProp offers the most statistically robust predictions for general monitoring, Adam provides better responsiveness to extreme events at the cost of precision. Future work will focus on dynamic switching mechanisms or refined Amnesia thresholds to leverage the strengths of both approaches.

VI. Conclusion

In this paper, we presented an edge-based online learning framework for real-time river water level prediction. By shifting the training process to the edge and utilizing a rolling normalization technique, we addressed two fundamental limitations of centralized forecasting systems: the high cost of data transmission and the rigidity of offline-trained models in dynamic environments.

Our experimental results reveal a distinct trade-off between generalization and adaptability. While traditional offline models (such as LSTM and GRU) produce smoother forecasts with lower global error, they fail to react sufficiently to sudden data shifts, often under-estimating critical flood peaks. In contrast, our online MLP model prioritizes recent data trends, allowing it to adapt immediately to the rising limb of a flood, providing a more effective tool for early warning despite slightly higher noise levels during stable periods.

Furthermore, our analysis of adaptive optimizers highlights that the choice of learning algorithm significantly influences this trade-off. We found that RMSProp offers superior stability for general monitoring, whereas Adam provides the sensitivity required for rapid peak detection. The experimental "Amnesia" strategy, while currently less stable, suggests that dynamic resetting mechanisms could further bridge the gap between stability and responsiveness. Future work will focus on refining these trigger mechanisms and exploring hybrid approaches to combine the robustness of recurrent networks with the adaptability of online learning.

References

[1] K. Smith and D. Petley, *Environmental Hazards: Assessing Risk and Reducing Disaster*. Routledge, 2009.

[2] UNISDR, "Global assessment report on disaster risk reduction," https://www.undrr.org, 2015.

[3] FAO, *The impact of disasters and crises on agriculture and food security*. Food and Agriculture Organization, 2017.

[4] F. Roda, J. Martinez *et al.*, "Global flood impacts on agriculture," *Environmental Research Letters*, vol. 14, no. 12, p. 123004, 2019.

[5] K. Beven, *Rainfall-Runoff Modelling: The Primer*. Wiley-Blackwell, 2012.

[6] V. T. Chow, D. R. Maidment, and L. W. Mays, *Applied Hydrology*. McGraw-Hill, 1988.

[7] Y. e. a. Tang, "Nonlinear hydrological processes and modeling," *Water Resources Research*, vol. 54, no. 5, pp. 3458–3480, 2018.

[8] IPCC, "Climate change 2021: The physical science basis," https://www.ipcc.ch/report/ar6/wg1/, 2021, intergovernmental Panel on Climate Change.

[9] A. Mosavi *et al.*, "Flood prediction using machine-learning models," *Sensors*, vol. 18, no. 11, p. 3435, 2018.

[10] C. Shu *et al.*, "Forecasting water levels using hybrid machine-learning models," *Journal of Hydrology*, vol. 583, p. 124627, 2020.

[11] F. Kratzert *et al.*, "Rainfall–runoff modelling with lstms," *Hydrology and Earth System Sciences*, vol. 23, pp. 5089–5110, 2019.

[12] X. Qin *et al.*, "Water-level forecasting using lstm networks," *Journal of Hydrology*, vol. 567, pp. 775–786, 2019.

[13] Z. e. a. Li, "Transformer-based hydrological modelling: A review," *Water*, vol. 15, no. 4, p. 655, 2023.

[14] Z. M. Yaseen *et al.*, "River water-level prediction using hybrid models," *Journal of Hydrology*, vol. 585, p. 124776, 2020.

[15] Y. Abadade, A. Temouden, H. Bamoumen, N. Benamar, Y. Chtouki, and A. S. Hafid, "A comprehensive survey on tinyml," *IEEE Access*, vol. 11, pp. 96 892–96 922, 2023.

[16] S. Shalev-Shwartz, "Online learning and online convex optimization," *Foundations and Trends in Machine Learning*, vol. 4, no. 2, pp. 107–194, 2012.

[17] T. Le *et al.*, "A stream learning approach for real-time hydrological forecasting," *Journal of Hydrology*, vol. 610, p. 127859, 2022.

[18] S. C. H. Hoi *et al.*, "Online learning: A comprehensive survey," *ACM Computing Surveys*, vol. 53, no. 5, pp. 1–36, 2021.

[19] F. Orabona, "A modern introduction to online learning," *arXiv:1909.05207*, 2019.

[20] B. Krawczyk, "Learning from imbalanced data streams," *Progress in Artificial Intelligence*, vol. 5, pp. 221–232, 2017.

[21] S. Riad, J. Mania, L. Bouchaou, and Y. Najjar, "Rainfall-runoff model using an artificial neural network approach," *Environmental Modelling & Software*, vol. 19, no. 4, pp. 375–386, 2004.

[22] M. Rezaeian Zadeh, S. Amin, D. Khalili, and V. P. Singh, "Daily outflow prediction by multi layer perceptron with logistic sigmoid and tangent sigmoid activation functions," *Water Resources Management*, vol. 24, no. 11, pp. 2673–2688, 2010.

[23] X.-H. Le, H. V. Ho, G. Lee, and S. Jung, "Application of long short-term memory (lstm) neural network for flood forecasting," *Water*, vol. 11, no. 7, p. 1387, 2019. [Online]. Available: https://doi.org/10.3390/w11071387

[24] M. N. A. Zakaria, A. N. Ahmed, M. Abdul Malek, A. H. Birima, M. M. Hayet Khan, M. Sherif, and A. Elshafie, "Exploring machine learning algorithms for accurate water level forecasting in muda river, malaysia," *Heliyon*, vol. 9, no. 7, p. e17689, 2023. [Online]. Available: https://doi.org/10.1016/j.heliyon.2023.e17689

[25] W. Almikaeel, A. Šoltész, L. Čubanová, and D. Baroková, "Hydroinformer: a deep learning model for accurate water level and flood predictions," *Natural Hazards*, vol. 121, no. 4, pp. 3959–3979, 2025.

[26] Y. Zhang, Z. Gu, J. V. Thé, S.-X. Yang, and B. Gharabaghi, "The discharge forecasting of multiple monitoring stations for humber river by hybrid lstm models," *Water*, vol. 14, no. 11, p. 1794, 2022.

[27] O. Anava, E. Hazan, S. Mannor, and O. Shamir, "Online learning for time series prediction," in *Proceedings of the 26th Annual Conference on Learning Theory*, ser. Proceedings of Machine Learning Research, S. Shalev-Shwartz and I. Steinwart, Eds., vol. 30. Princeton, NJ, USA: PMLR, 12–14 Jun 2013, pp. 172–184. [Online]. Available: https://proceedings.mlr.press/v30/Anava13.html

[28] J. Pardo, F. Zamora-Martínez, and P. Botella-Rocamora, "Online learning algorithm for time series forecasting suitable for low-cost wireless sensor networks nodes," *Sensors*, vol. 15, no. 4, pp. 9277–9304, 2015.

[29] S. C. Hoi, D. Sahoo, J. Lu, and P. Zhao, "Online learning: A comprehensive survey," *Neurocomputing*, vol. 459, pp. 249–289, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231221006706

[30] E. Ogasawara, L. C. Martinez, D. de Oliveira, G. Zimbrão, G. L. Pappa, and M. Mattoso, "Adaptive normalization: A novel data normalization approach for non-stationary time series," in *The 2010 International Joint Conference on Neural Networks (IJCNN)*, 2010, pp. 1–8.

[31] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.

[32] T. Tieleman and G. Hinton, "Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude," COURSERA: Neural networks for machine learning, pp. 26–31, 2012.

[33] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*, 2019.

[34] L. Liu, H. Jiang, P. He, W. Chen, X. Liu, J. Gao, and J. Han, "On the variance of the adaptive learning rate and beyond," in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*, 2020.

[35] E. Nikishin, M. Schwarzer, P. D'Oro, P.-L. Bacon, and A. Courville, "The primacy bias in deep reinforcement learning," in *Proceedings of the 39th International Conference on Machine Learning*. PMLR, 2022, pp. 16 828–16 847.

[36] I. Loshchilov and F. Hutter, "Sgdr: Stochastic gradient descent with warm restarts," in *International Conference on Learning Representations (ICLR)*, 2017.